Öğrenci:
Mehmet Efe Gezici
E-Posta:
256001060@stu.adu.edu.tr

Ders:
Web Programcılığı III - BPR201
Öğretim Elemanı:
Öğr. Gör. Mehmet Can Hanaylı

OOP Prensipleri Araştırma Ödevi

Bu ödevde nesne tabanlı programlamanın (OOP) mantığını kopyala-yapıştır tanımlarla değil; oyunlardan, günlük hayattan örneklerle ve kendi cümlelerimle açıklamaya çalıştım. Amacım bu yapıların kod yazarken bizi hangi dertlerden kurtardığını göstermek.

1. Sınıf (Class) ve Nesne (Object)

Sınıf bir şeyin üretim taslağıdır, nesne ise o taslaktan bellekte üretilmiş gerçek varlıktır.

> **Oyun Örneği:** Minecraft'taki "Kılıç Tarifi" bir sınıftır (Class). O tarifi kullanarak ürettiğiniz ve envanterinize gelen "Elmas Kılıç" ise bir nesnedir (Object).
> **Çözdüğü Problem:** Özellikleri (kılıcın hasarı) ve eylemleri (saldırma) tek bir pakette toplamamızı sağlar. Her kılıç için ayrı ayrı değişken tanımlama ameleliğinden kurtarır.

```
class Kilic {
    int hasar; // Özellik
    void Saldir() { /* Düşmana vur */ } // Eylem
}
// Nesne oluşturma (Elmas kılıcı ürettik)
Kilic elmasKilic = new Kilic();
elmasKilic.hasar = 10;
```

2. Encapsulation (Kapsülleme)

Sınıf içindeki verileri (değişkenleri) korumaya alıp, dışarıdan kafaya göre değiştirilmesini engellemektir.

> **Neden Kullanıyoruz?:** Örneğin Valorant oynarken karakterinizin can değerini dışarıdan direkt can = 9999 diye değiştiremezsiniz (hile olur). Can sadece hasarAl() veya canBas() metotlarıyla, kurallara uygun değişebilir. Bu sayede oyunda (veya projede) bug oluşması engellenir. Değişkenleri gizli (private) tutup ipleri elimizde tutarız.

```
class Karakter {
    private int can = 100; // Dışarıdan doğrudan erişilemez!

    public void HasarAl(int miktar) {
        if(miktar > 0) { can -= miktar; } // Sadece kurala uyan erişim
    }
}
```

3. Abstraction (Soyutlama)

Arka plandaki gereksiz ve karmaşık detayları gizleyerek, sadece işimize yarayan basit kısmı öne çıkarmaktır.

> **Günlük Örnek:** PS5 kolunda "X" tuşuna basınca karakterin zıpladığını biliriz. O tuşa basınca arkada Bluetooth sinyali nasıl gitti, işlemci bunu nasıl okudu ilgilenmeyiz.
> **Çözdüğü Problem:** Bizden sonra kodu kullanacak kişilerin kafasını karıştırmaz. "Sen sadece Zıpla metodunu çağır, arkasını boşver" deriz.

```
abstract class KonsolKolu {
    // İçeride ne döndüğü bizi ilgilendirmez, şablonu verir
    public abstract void TusaBas(string tus);
}
class PS5Kolu : KonsolKolu {
    public override void TusaBas(string tus) { /* Bluetooth işlemleri vs. */ }
}
```

4. Inheritance (Kalıtım)

Alt sınıfların, bir ana sınıfın özelliklerini ve yeteneklerini hazırdan miras almasıdır.

> **Oyun Örneği:** RPG oyunlarında genel bir "Karakter" sınıfı vardır. Hem "Savaşçı" hem de "Okçu" bu sınıftan miras alır. İkisinin de yürüme, can, mana gibi ortak özellikleri Karakter'den gelir.
> **Çözdüğü Problem:** Ortak kodları tekrar tekrar yazmaktan (kod hamallığından) bizi kurtarır.

```
class Karakter {
    public int can;
    public void Yuru() { /* ... */ }
}
class Savasci : Karakter {
    // can ve Yuru() özelliklerini bedavaya aldı
    public void KilicSalla() { /* Sadece savaşçıya özel */ }
}
```

5. Polymorphism (Çok Biçimlilik)

Aynı isimli bir metodun, miras alan farklı sınıflarda farklı şekillerde çalışabilmesidir.

> **Mantığı:** Kalıtım örneğindeki Savaşçı ve Okçu'yu düşünün. İkisinin de Saldir() metodu vardır (ana sınıftan gelir). Ama Savaşçı kılıç savururken, Okçu yay gerer. Aynı komut, farklı aksiyon doğurur. Projeye yeni bir "Büyücü" eklediğimizde ana sistemi hiç bozmadan yeni bir saldırı şekli ekleyebiliriz.

```
class Dusman {
```

6. Constructor (Yapıcı Metot) ve Nesne Oluşturma

Bir nesne "new" kelimesiyle yaratıldığı ilk anda, saniyesinde otomatik çalışan hazırlık metodudur.

> **Günlük Örnek:** Yeni bir akıllı telefon aldınız ve ilk kez güç tuşuna bastınız. Karşınıza "Dil Seçimi" ve "Wi-Fi Bağlantısı" ekranı çıkar. Bunları yapmadan telefonu kullanamazsınız.
> **Çözdüğü Problem:** Nesnenin bellekte saçma sapan veya eksik verilerle oluşmasını engeller. Başlangıç (kurulum) şartlarını belirler.

```
class Oyuncu {
    public string Nickname;
    // Sınıfla aynı isimde olur, geri dönüş tipi yoktur
    public Oyuncu(string nick) {
        this.Nickname = nick; // Başlangıçta nick vermek zorunlu!
    }
}
Oyuncu p1 = new Oyuncu("Faker");
```

7. Access Modifiers (Erişim Belirleyiciler)

Değişkenlere ve metotlara "Kimler ulaşabilir?" sorusunun cevabıdır.

> **Nasıl Ayırıyoruz?:**
> • **Public:** Herkesin girdiği halka açık park. Her yerden ulaşılır.
> • **Private:** Kilitli günlüğünüz. Sadece sizin (o sınıfın) haberi olur, dışarısı göremez.
> • **Protected:** Aile yadigarı köstekli saat. Sadece siz ve çocuklarınız (miras alan sınıflar) kullanabilir.

8. Interface ve Abstract Class Kavramları

İkisi de şablon oluşturur ama kullanım amaçları tamamen farklıdır.

> **Kısaca Farkları:**
> • **Abstract Class (O bir şudur):** "Araba" bir abstract class'tır. Bir şey aynı anda hem Araba hem Uçak olamaz (sadece tek bir class'tan miras alınır). Şablonun bir kısmı doldurulmuş olabilir.
> • **Interface (Şunu yapabilir):** "TurboKullanabilme" bir interface'tir. Bir sözleşmedir, içi boştur sen doldurursun. Bir nesneye birden fazla interface ekleyebilirsin.

```
interface IUcabilir { void Uc(); } // Yapılması zorunlu kural
class PelerinliKahraman : IUcabilir {
    public void Uc() { /* Kendi uçma kodunu yazmak zorunda */ }
}
```

9. Composition ve Inheritance Arasındaki Fark

Yazılım dünyasındaki en büyük ikilemlerden biridir. Sırf kod tekrarı olmasın diye her şeyi miras almak (Inheritance) mantıklı değildir.

> **Fark Nedir?:**
> • **Kalıtım (Inheritance):** "is-a" ilişkisidir. Otomobil bir Taşıttır.
> • **Kompozisyon (Composition):** "has-a" ilişkisidir. Otomobilin bir Motoru vardır.
> **Hangisi İyi?:** Eğer kalıtımı çok abartırsak kod spagettiye döner. Kompozisyon ise parçaları Lego gibi birbirine takıp çıkarmaktır. Mesela silaha dürbün takmak istiyorsak, yeni bir "DürbünlüSilah" sınıfı türetmek yerine, Silah'ın içine bir Dürbün objesi eklemek (Composition) çok daha akıllıcadır.

```
// Kompozisyon (Mantıklı olan yapı)
class Durbun { public void Yaklas() { } }

class KeskinNisanciTufegi {
    private Durbun scope = new Durbun(); // İçinde bir dürbün objesi taşıyor
    public void NisanAl() { scope.Yaklas(); }
}
```

10. Nesneler Arasındaki İlişkiler

Gerçek bir projede hiçbir nesne tek başına takılmaz. Birbirleriyle iletişim kurarlar.

> **Bağlantı Türleri:**
> • **Normal İlişki:** İkisi de bağımsızdır. Öğrenci ve Öğretmen gibi.
> • **Aggregation (Gevşek Bağ):** Okul ve Öğrenci gibi. Okul yıkılsa bile öğrenciler hayatta kalır, yok olmazlar.
> • **Composition (Sıkı Bağ):** İnsan ve Kalp gibi. İnsan nesnesi silinirse, Kalp nesnesi de mecburen silinmelidir, tek başına yaşayamaz. Bu yüzden kodu yazarken birbirlerine ne kadar bağımlı olduklarını doğru kurgulamak gerekir.

Sonuç ve Özet

Sonuç olarak hocam, OOP bizim için sadece süslü kod yazma sanatı değil, gerçek dünyadaki sorunları bilgisayara anlatma yöntemimizdir. Doğru kullanıldığında projemiz ne kadar büyürse büyüsün; yeni bir silah, yeni bir karakter veya yeni bir sistem eklemek eski kodları bozmadan kolayca halledilebilir.